Seminar Paper
On Ends-to-Ends Encryption: Asynchronous Group Messaging with Strong Security Guarantees
by Cohn-Gordon et al.

Seminar
Cryptography in Secure Messaging: Understanding and Enhancing Signal
by Paul Roesler

Johan Zenk
he89fabi

January 2023

Abstract

This seminar paper summarizes the core ideas behind *On Ends-to-Ends Encryption: Asynchronous Group Messaging with Strong Security Guarantees by Cohn-Gordon et al.* [4]. Its main contribution is a new design called *Asynchronous Ratcheting Tree* (ART) that enables secure group messaging and provides as first of its kind Post-Compromise Security (PCS) and logarithmic scaling in the group size. After first introducing to prior related work *Ratcheted Encryption and Key Exchange: The Security of Messaging by Bellare et al.* [1], and defining objectives and security properties of ART, this paper aims to break down and summarize the key concepts of ART and to provide a fundamental understanding of its core novelties. In the end both papers will be briefly compared and a prospect of related later work is given regarding Continous Group Key Agreement (CGKA) protocols.

Contents

1 Prior Related Work **3**

2 Introduction **6**

3 Background **7**

3.1 Objectives and Security Properties 7

3.2 Tree-based group Diffie-Hellman 8

4 Design of Asynchronous Ratcheting Tree (ART) **9**

4.1 Construction 9

4.2 Key Updates 11

4.3 Comparison 12

5 Conclusion and Later Work **14**

Motivation

Many modern messaging systems use end-to-end encryption providing secure communication for over a billion active users. Especially *ratcheting* has become quite famous, which regularly updates the key during communication promising strong theoretically cryptographic properties. Because of this, a lot of research is going on in recent years discussing and improving this technique.

1 Prior Related Work

In this section I want to briefly emphasize the idea of prior similar work by Bellare et al., which is *Ratcheted Encryption and Key Exchange: The Security of Messaging* [1]. Relative to this prior work I additionally want to highlight the core novelties that Cohn-Gordon et al. are introducing in their paper.

In Bellare et al. [1] single, one-sided ratcheting is investigated within a two-party communication setup. Furthermore, a formal representation of the ratcheting technique as a cryptographic primitive is provided by introducing a precise syntax for encryption and key exchange. On top of that, a protocol for the key exchange based on Diffie-Hellman (DH) is proposed and its corresponding security goals are formalized. These security goals are namely: forward security, post-compromise-security, authenticated encryption. The paper's contributed model proposes a general and flexible method that allows to do *ratcheting* at any time and is proven to be secure within a range of assumptions.

The Ratcheted Key Exchange allows users to agree on a shared secret with strong security guarantees. In the paper the communication is simplified with a one-sided communication scheme introducing the participants as "sender" and "receiver" which can be easily extended analogously to multiple-party communication.
At first an initial key algorithm IKg generates and distributes a set of keys $k, stk_s, stk_r, sek_s, sek_r$ where k is an initial shared secret that can be used by both parties for any purpose, i.e. for symmetric encryption of messages. stk_s and stk_r are long-term keys that are not updated. stk_s is known to all parties, but stk_r is only known by the receiver. The session keys sek_s and sek_r are required for future key exchanges, f.e. MAC keys to authenticate the key exchange.
The sender gets the keys stk_s, sek_s, k_s and the receiver the keys stk_s, stk_r, sek_r, k_r. The equation $k_s = k_r = k$ is valid if an attacker didn't successfully tampered with the protocol. "r" and "s" are defined as subscripts to define output keys and session keys and indicate if the keys belong to the sender or the receiver.
Next the sender's and receiver's key generation algorithms (SKg and RKg) are defined which model the key ratcheting process to produce new session keys upon deleting the old keys: If the sender wants to produce a new shared secret, the algorithm takes the sender's static key stk_s and its session key sek_s to generate a new session key sek_s and output key k_s. The resulting update information upd enables the receiver to generate the same result: The algorithm takes the sender's static key stk_k, its own static key stk_r, its session key sek_r, the received update information upd and the current output key k_r. The return values contain the new session key sek_r, output key k_r and a bool-flag indicating whether the key generation was successful or not:

upd could therefore be easily rejected which results in not updating the keys.
The interaction between these algorithms is visualized in the following figure (1).

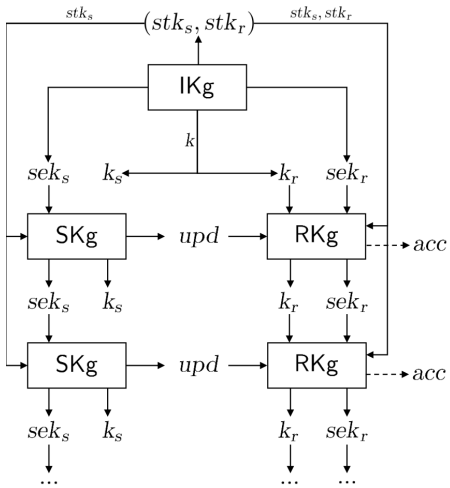

Figure 1: The interaction between ratcheted key exchange algorithms [1]

The security of ratcheted key exchange should be strong. An active attacker is assumed that can intercept and modify all messages going from the sender to the receiver. The goals demand that an attacker should not be able to distinguish produced output keys from random strings and cannot make both parties agree on output keys that do not match. On top of that even if an adversary has stolen the secrets stored by the sender, it should not be able to reveal secrets from prior and future communication or to say: Not be able to distinguish those messages from random strings. These goals are defined as forward secrecy and Post-Compromise Security, which I will later explain in detail.

Given the short overview above and a well-formulated summary of [6] I briefly want to sketch a generic example for a key update performable by any party.
Any party's internal state is defined by (i, K, Y) where i is a counter for the send operations done, K is a key for a Pseudo-Random Function PRF and Y is the other parties public key. When performing a send operation first a fresh randomness x is sampled and the value $\mu = PRF(K, g^x)$ is created. Using a random oracle H a newly established key k and the next round's key K' for the PRF is computed as follows $(k, K') = H(i, \mu, g^x, Y^x)$ leading to the new subsequent state $(i + 1, K', Y)$. The other party will be informed by an update message (g^x, μ) where μ serves as a Message Authentication Code (MAC). Note that F and H together form a hash chain that will provide forward security (FS). The inputs g^x and Y^x of H can be seen as one Diffie-Hellman-Key-Exchange per transmission with one exponent being static.

Ratcheted Encryption uses the secrets shared formerly to encrypt messages: The algorithm builds up a symmetric encryption scheme that takes the output key k_s as an encryption key, a nonce n, the plaintext message m and a header h to return the ciphertext c. The receiver can use its output key k_r (remember: ideally $k_r = k_s$ is valid), the same nonce n and header h to

decrypt the ciphertext c to receive the plaintext message m. Note that this encryption scheme is a nonce-based authenticated encryption, so authenticity is guaranteed if nonces are not misused.

So in summary Bellare et al. proved formally the Ratcheted Encryption and Key Exchange to be secure under their assumptions. In fact this scheme can also be used in secure group communication where each member has a secure pairwise channel with any other member. Note that this scheme would result in linear scaling w.r.t the group size because when a member wants to broadcast a message, this message has to be copied, encrypted and sent over each pairwise channel separately. But also the key exchange and further updates had to be done with each group member individually. This is infeasible for large group sizes, i.e. groups with more than 1,000 members.

Given this problem regarding group communication I now want to segue into the work of Cohn-Gordon et al. [4] where they propose a design for secure, asynchronous group communication that scales sub-linearly in the group size and provides strong security properties such as Forward Secrecy and Post-Compromise Security.

In section 4.3 I will highlight a few similarities and differences between the 2-party Ratcheted Key Exchange briefly described above and the scheme by Cohn-Gordon et al.

2 Introduction

There has been a lot of research in group messaging for decades and especially those of interest that provide secure end-to-end communication via some kind of Group Key Exchange Protocol. The main difference between this paper relative to the former work is that it focuses on asynchronicity and Post-Compromise-Security (PCS) in secure group messaging. The special thing about the proposed design called ART is that it can derive a group key with PCS even if no two participants are online at the same time which captures the essence of asynchronicity. This is extremely valuable especially in unreliable networks and in networks where bandwidth is limited. In general when the paper was released most of the messaging systems did provide Forward Security (FS) and PCS, but had dropped the latter security property without notification to its users if the communication had been expanded to 3+ users. In subsection 3.1 a detailed definition of FS and PCS is given.
As group and multi-device messaging has become mainstream for many users, but the communication has to be asynchronous, it is not straightforward to adapt existing group key exchange protocols, which usually require several interactive rounds of communication. However one could establish two-party protocols like 2-party Ratcheting between every pair of group members like described in the prior section. This is seen as inefficient and infeasible regarding larger group sizes.

The main research question of this paper is: Is there a secure group messaging solution that...

(i) ... allows participants to communicate asynchronously, ...

(ii) ... scales sub-linearly in the group size and ...

(iii) ... admits strong security guarantees such as PCS ?

3 Background

First I want to give a short overview of the tree-based group Diffie-Hellman (DH), which will be the baseline technique to sketch the ART design. In addition objectives and security properties will be discussed.

3.1 Objectives and Security Properties

Secrecy and Authentication The fundamental goals are confidentiality and authenticity of all keys, so an adversary should not be able to learn any secrets shared between any two participants.

Security under an network adversary It is assumed that an adversary has full control of message delivery and is able to intercept, read and modify any messages over the network. With this assumption all security goals, namely confidentiality, integrity and availability, should not be violated, but of course the latter one cannot be guaranteed when an adversary decides to stop delivery.

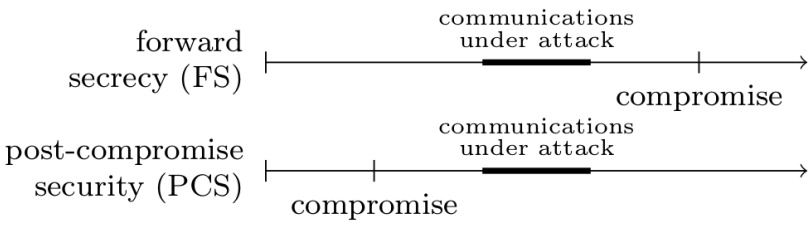

Figure 2: time lines defining the scenarios of FS and PCS [4]

Forward Secrecy (FS) *"protection against later compromise"*. This is also known as perfect forward secrecy (PFS) and a property that assures that prior secrets cannot be unrevealed by keys or random values learned within subsequent communication.

Post-Compromise Security (PCS) *"protection against earlier compromise"*. This property is not provided by traditional security models as they do not provide any guarantees after long-term key material of a participant was compromised. An adversary can then impersonate and eavesdrop on the affected participants. [3] defined PCS, showing that it is achievable through the use of a persistent protocol state. ART aims to achieve PCS: if a member in the group is temporarily compromised, then future group communication and its resulting group key should be secret again. Because of this reason, PCS is sometimes referred to as "self-healing". The Sender Keys variant of Signal has the weakness that once the sender key is learned by an adversary, all other future message keys are derivable and all messages can be decrypted. PCS offers significant protection from adversaries because they are forced to actively interfere in all communications to intercept group messages indefinitely.

3.2 Tree-based group Diffie-Hellman

The idea of this scheme, which is later also used to propose ART's design, is to assign private DH keys at the leaves of a binary tree. Two leaf nodes, that share the same direct parent node, have the secret keys x and y and their respective public keys g^x and g^y. They now can do the usual DH key exchange resulting into their shared secret g^{xy}. This secret is assigned to their parent node and functions as its secret key, therefore its resulting public key is $g^{g^{xy}}$. Following this pattern recursively this yields a value at the tree's root called the "tree key".
Any agent who wants to compute the secret $g^{xy} = (g^y)^x = (g^x)^y$ of a non-leaf node, must know the secret key x (or y) of one of its children and the corresponding public key g^y (or g^x) of the other child.
The tree key tk can only be computed with the knowledge of at least one secret leaf key and all public node keys pk_1 to pk_n along its co-path, which is the set of all siblings nodes along its path to the root of the tree. By repeatedly exponentiating the keys along the path, an agent can derive the tree key value at the root.

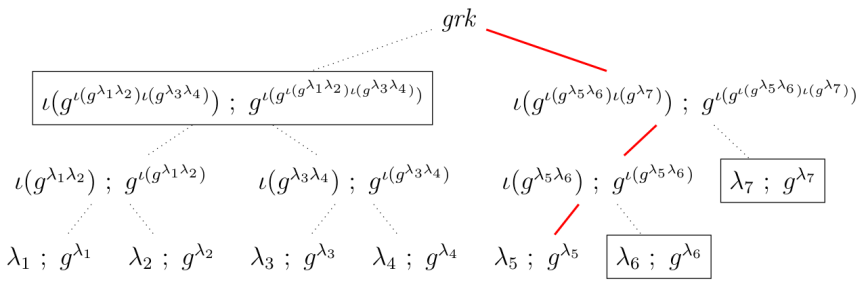

Figure 3: example of a Diffie-Hellman tree [4]

Figure (3) shows an example Diffie-Hellman tree. Each node is marked with its secret and its corresponding public key, separated by **;** .
λ_j denotes a secret leaf key and $\iota(.)$ defines a mapping from group elements to integers.
The path from λ_5 to the root is marked in red and the nodes within its co-path are indicated with a surrounding box.
If the owner of λ_5 wishes to obtain the tree key at the root node, it has to know the public keys g^{λ_6}, g^{λ_7} and $g^{\iota(g^{\iota(g^{\lambda_1\lambda_2})\iota(g^{\lambda_3\lambda_4})})}$ to compute it.

4 Design of Asynchronous Ratcheting Tree (ART)

4.1 Construction

The reason why DH-key-exchange cannot be applied directly is because the initiator of the group, let's call her Alice (A), may be the only person to be online at the time of creation. As the algorithm is a synchronous key exchange protocol, it needs all participants to be online simultaneously. For example in a group with four members, even if A would have all the public keys of each member, she would not be able to compute all the keys necessary to compute the shared secret group key at the root of the tree.

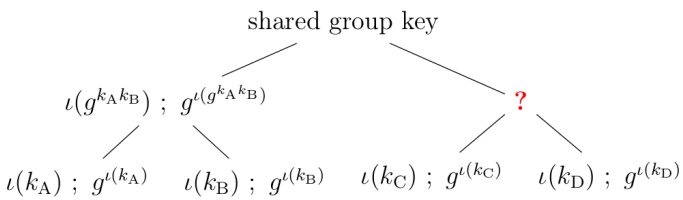

Figure 4: when A creates a initial group tree and C and D are offline, A cannot create the shared group key because the public key at its co-path is missing (marked with the red ?) [4]

Because of this issue, a design is proposed in which the secret keys of a leaf node can be derived with 2 important properties:
1. for each group member, the initiator and the group member can asynchronously derive that key (no need to be online at the same time)
2. no other actors, either group members or an adversary, can derive that key.
This proposal sure leads to additional trust assumptions on the initiator, but these can be circumvented by introducing an initial key update by each member of the group before the actual communication starts. Key updates are explained more detailed in the next section.

So how can these leaf keys be derived? The answer is that these keys can be the session keys of any strong one-round Authenticated Key Exchange (AKE) protocol which will be denoted as KEY-EXCHANGE and takes two private keys (ik and ek) and their two corresponding public keys (IK and EK) and returns a bitstring β that has the following property:

$$\beta = \text{KEY-EXCHANGE}(ik_1,\ IK_2,\ ek_1,\ EK_2) = \text{KEY-EXCHANGE}(ik_2,\ IK_1,\ ek_2,\ EK_1)$$

In words this enables any two parties (1 and 2) to produce the exact same result β using their own private key together with the other party's public key. ART does not have any further requirements on the structure of the AKE protocol used.

To establish such a design the idea of prekeys is used. Prekeys are ephemeral public keys, stored temporarily by a server and fetched on demand by messaging clients via the Public-Key-Infrastructure. At the same time long-term identity keys are delivered.

When a group is created the first thing which has to be done is to compute an initial tree. A setup key-pair is introduced that is generated locally by the creator and used only during creation to perform an initial key exchange with the prekeys. This allows the initiator to generate secret leaf keys for any other group member while they are offline.

So if A creates a new group with a size of n, containing herself and n-1 peers, A starts by generating the DH setup key pair $suk, SUK = g^{suk}$. Afterwards A requests from the PKI the public identity keys IK and ephemeral prekeys EK for each of its intended peers ranging from 1 to n-1. These public keys are computed together with A's secret identity key ik_A and the private setup key suk to establish an one-round key exchange protocol to derive the leaf keys $\lambda_1, ..., \lambda_{n-1}$.

Because these leaf keys will be later updated, they are notated as λ_i^u with i marking a counter and u the corresponding identity.

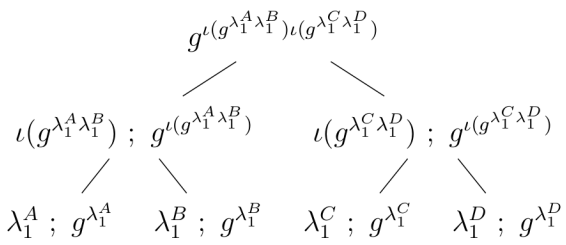

Figure 5: freshly generated ART tree [4]

In Figure (5) we see an initial tree for 4 group members (A, B, C and D).

Assuming A has initiated the group it computes the leaf keys as follows:
λ_1^B = KEY-EXCHANGE($ik_A,\ IK_B,\ suk,\ EK_B$)
λ_1^C = KEY-EXCHANGE($ik_A,\ IK_C,\ suk,\ EK_C$)
λ_1^D = KEY-EXCHANGE($ik_A,\ IK_D,\ suk,\ EK_D$)
A creates its own fresh leaf key λ_1^A and then recursively constructs a DH-tree whose root is the shared secret or the initial group tree key.
Afterwards A is ready to share the information with the other group members allowing them to reproduce the same results. For this A broadcasts the following material:

(i) the public prekeys (EK_i) and identities (IK_i) that A used

(ii) the public setup key SUK = g^{suk}

(iii) the tree T of public keys A derived

(iv) a signature of (i), (ii), (iii) under A's private identity key ik_A

Each member should be able to extract the same tree key. So after receiving the broadcasted message and verifying its signature (using IK_A), any member u $\in \{B, C, D\}$ can first compute its initial leaf key

$\lambda_1^u =$ KEY-EXCHANGE$(ik_u,\ IK_A,\ ek_u,\ SUK)$
and then compute the tree key by first extracting their co-path's public keys and then iteratively exponentiate them until they reach the root. This tree key at the root is the shared secret among all members.

4.2 Key Updates

Any member of the group should be able to update their key using an efficient mechanism. One of the main goals of ART is to achieve PCS, so it has to satisfy the requirement that the stage keys cannot be independent, instead each stage key must depend on both, the newly exchanged messages and the state from previous stages. As long as one of these two sources is unknown to an adversary, the resulting stage key will be as well. Cohn-Gordon, Cremers, and Garratt prove the necessity of this double dependency in their work[2].

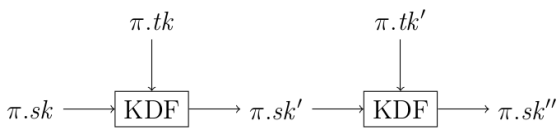

Figure 6: derivation of the stage keys $\pi.sk$, stage key chaining [4]

In ART, the resulting stage keys form a hash chain which is depicted in Figure (6). The new stage key $\pi.sk'$ is computed with the current stage key $\pi.sk$ and the newly computed tree key $\pi.tk$. The Key-Derivation-Function (KDF) of ART also takes as input: The group members' ordered identifiers $\pi.IDs$ and the current tree $\pi.T$ with ordered nodes that hold the public keys.

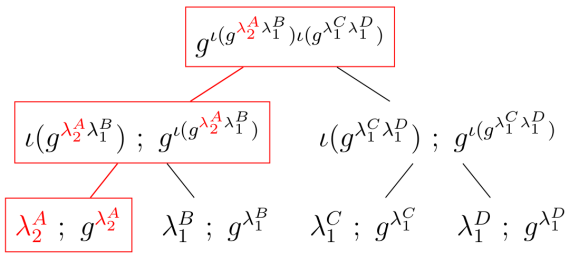

Figure 7: member A updates its key and all the keys up to the root [4]

If any member, i.e. A wants to update its secret leaf key from λ_1^A to λ_2^A, it first has to re-compute all the public DH keys at the nodes along its path to the root of the tree. Because of the logarithmic depth of a tree, this can be done in logarithmic time regarding the group size and is therefore efficient compared to the pairwise-ratcheting approach. After the public keys had been computed, A broadcasts its public leaf key $g^{\lambda_2^A}$ together with the other re-computed public keys to the group members. This message will be authenticated with a Message Authentication

Code (MAC) under a key derived from the previous stage key.
Any group member can verify this message, because it knows how to derive the key used for the MAC, and locally updates their stored co-path in the tree with the received public keys. Finally the updated group key at the root can be retrieved. Figure (7) shows the described scenario where A updates its key.

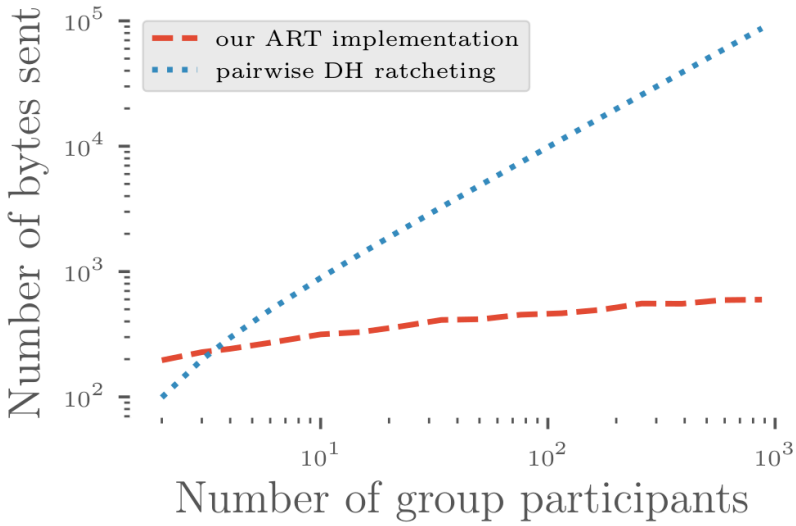

Figure 8: outgoing bytes needed to sent a 32-byte message [4]

We see in figure (8) the concept of ART scales much better than the pairwise approach. This plot had been created after the protocol was implemented and benchmarked to the pairwise ratcheting approach. Note that both axes are logarithmically scaled which underlines the advantage of ART additionally.

4.3 Comparison

In this subsection the group ART scheme is compared with the two-party Ratcheted Key Exchange scheme described in 1.
Of course both of them have in common that they provide PCS by enabling any party to do key updates using fresh randomness. While ART does not specify exactly how often these updates happen, the pairwise protocol rotates the key whenever a send operation is performed.
We have seen that in the 2-party Ratcheted Key-Exchange RKE the new key is generated using a PRF and a random oracle while in ART a key-derivation function is used. Both approaches use the current state to produce a new state and the corresponding stage key. These resulting stage keys form a hash chain that provide FS. Because ART is operating in a group setup, during a key update it also has to consider all the member identity keys along with all the current public keys.
After updating its key in ART, a member not only has to send one public key via the update information, instead all updated public keys along its path to the root of the tree have to be

announced.
Both schemes use a MAC to secure the update messages from being tampered with. In general the key used to generate this MAC is derived from the current stage key and is therefore also renewed periodically.

5 Conclusion and Later Work

In summary *Cohn Catriel et al* showed that ART allows asynchronous group communication, providing strong security properties such as FS and PCS and that their design scales only logarithmically with the group size that promises large group sizes.

In general Continuous group key agreement (CGKA) is at the core of Secure Group Messaging (SGM) and captures the basic primitive underlying group messaging schemes and "allows a group of users to maintain a continuously updated shared key in an asynchronous setting where parties only come online sporadically and their messages are relayed by an untrusted server." [11]

After ART many other CGKA protocols had been published, most of them rely on a basic tree structure as these tree structures provide logarithmic depth. Most notable are those protocols from the TreeKEM family, the simplest of which is used in the Messaging Layer Security (MLS), a protocol designed by the IETF working group. TreeKEM is similar to ART where a group is still mapped to a balanced binary tree where each node is assigned a public and secret key but does not require any relation between the secret key of a node and the secret keys of its parent nodes [7] [5]. Further improvements on TreeKEM regarding efficiency were done in Tainted TreeKEM (TTKEM) [7].[8] provided next to improvements a dedicated security analysis for it.

Later *Bienstock et al* remark in their article [9] that the communication protocols developed in the context of IETF's MLS do reduce the effort of sending a message to a logarithmic scale, but only with the restriction that group members are not allowed to send and recover from exposures concurrently which means that reaching PCS is delayed up to at least n communication time slots. Again for larger group sizes, this is just too expensive. In [4] *Cohn-Gordon et al* do mention concurrent key updates explicitly when designing ART, but these were considered the out-of-scope for their work. Anyway, they declared this problem as a future research question.

In [10] *Alwen et al* propose a design called Concurrent Continuous Group Key Agreement (CoCoA) to tackle the problem regarding concurrent key updates. They showed that using CoCoA all previously compromised users will recover after at most $log(n)$ update rounds with group size n. Additionally CoCoA is a server-aided CGKA to keep communication complexity low, as it no longer blindly forwards the packets to the users.

Alwen et al even go further and provide a 'fast-healing' concurrent CGKA protocol named *Decentralizable Continuous Group Key Agreement with Fast Healing* DeCAF [11]. Users can regain security after at most $log(t)$ requests with t being the number of corrupted users and therefore "heal" faster than in CoCoA. Protocol messages such as add/remove/update are posted on a blockchain instead of sent to a server. The downside of DeCAF is that the server-user communication is larger than in CoCoA because every user in DeCAF has to process the update of every other user.

References

[1] Mihir Bellare et al. *Ratcheted Encryption and Key Exchange: The Security of Messaging*. Cryptology ePrint Archive, Paper 2016/1028. https://eprint.iacr.org/2016/1028. 2016. URL: https://eprint.iacr.org/2016/1028.

[2] Katriel Cohn-Gordon, Cas Cremers, and Luke Garratt. *Post-Compromise Security*. Cryptology ePrint Archive, Paper 2016/221. https://eprint.iacr.org/2016/221. 2016. DOI: 10.1109/CSF.2016.19. URL: https://eprint.iacr.org/2016/221.

[3] Katriel Cohn-Gordon et al. *A Formal Security Analysis of the Signal Messaging Protocol*. Cryptology ePrint Archive, Paper 2016/1013. https://eprint.iacr.org/2016/1013. 2016. URL: https://eprint.iacr.org/2016/1013.

[4] Katriel Cohn-Gordon et al. *On Ends-to-Ends Encryption: Asynchronous Group Messaging with Strong Security Guarantees*. Cryptology ePrint Archive, Paper 2017/666. https://eprint.iacr.org/2017/666. 2017. URL: https://eprint.iacr.org/2017/666.

[5] Karthikeyan Bhargavan, Richard Barnes, and Eric Rescorla. *TreeKEM: Asynchronous Decentralized Key Management for Large Dynamic Groups A protocol proposal for Messaging Layer Security (MLS)*. Research Report. Inria Paris, May 2018. URL: https://hal.inria.fr/hal-02425247.

[6] Bertram Poettering and Paul Rösler. *Asynchronous ratcheted key exchange*. Cryptology ePrint Archive, Paper 2018/296. https://eprint.iacr.org/2018/296. 2018. URL: https://eprint.iacr.org/2018/296.

[7] Joël Alwen et al. *Keep the Dirt: Tainted TreeKEM, Adaptively and Actively Secure Continuous Group Key Agreement*. Cryptology ePrint Archive, Paper 2019/1489. https://eprint.iacr.org/2019/1489. 2019. URL: https://eprint.iacr.org/2019/1489.

[8] Joël Alwen et al. *Security Analysis and Improvements for the IETF MLS Standard for Group Messaging*. Cryptology ePrint Archive, Paper 2019/1189. https://eprint.iacr.org/2019/1189. 2019. URL: https://eprint.iacr.org/2019/1189.

[9] Alexander Bienstock, Yevgeniy Dodis, and Paul Rösler. *On the Price of Concurrency in Group Ratcheting Protocols*. Cryptology ePrint Archive, Paper 2020/1171. https://eprint.iacr.org/2020/1171. 2020. URL: https://eprint.iacr.org/2020/1171.

[10] Joël Alwen et al. *CoCoA: Concurrent Continuous Group Key Agreement*. Cryptology ePrint Archive, Paper 2022/251. https://eprint.iacr.org/2022/251. 2022. URL: https://eprint.iacr.org/2022/251.

[11] Joël Alwen et al. *DeCAF: Decentralizable Continuous Group Key Agreement with Fast Healing*. Cryptology ePrint Archive, Paper 2022/559. https://eprint.iacr.org/2022/559. 2022. URL: https://eprint.iacr.org/2022/559.